



Projet socializus

2026

React Native web - Nestjs - mongoDB
OVH - instance Public cloud - ubuntu

Sommaire	2
I - LA JOURNÉE D'INTÉGRATION	5
1. Principe des réunions asynchrones	5
2. Outils de communication	5
a - WhatsApp	5
b - Téléphone	6
c - Google Meet	6
3. Protocoles de communication et de travail quotidien en télé présentiel	6
a - Règles importantes	6
4. Environnement de travail — Chrome et compte professionnel	7
Utilisation d'un utilisateur Chrome "partagé"	7
Utilisation d'un utilisateur Chrome "pro"	7
Règles importantes	7
Adresse email de récupération	7
5. Gestion des mots de passe	8
6. Prises de notes — Google Drive et Google Docs	8
7. Le conducteur d'intégration	8
8. La documentation	8
9. Outils techniques complémentaires	9
Google Remote Desktop	9
Loom	9
Winscribe / VPN (Hola)	9
10. Posture et engagement attendus	9
II - DOCUMENTATION, ORGANISATION ET IA 2026	11
1. Objectif de la documentation	11
2. Organisation générale du projet	11
III Architecture du code 2026	12
6) Historique et suivi	12
7) Front-end	12
8) Back-end	12
9) Tests	12
12) Assets et ressources	12
13) Intégration développeur	12
10) Gestion des bugs	12
3) Évolutions par rapport à 2024	13
4) Convention de nommage	13
5) Structure des écrans	13
11) Authentification et sécurité	13
14) Résumé des changements majeurs	13
15) Restructuration de l'architecture du code en mode responsive.	14
	2

V Compilation 2026	16
Contexte & Objectif	16
Environnement de travail & Outils	16
Architecture du Code	16
Stratégie Mobile & WebView	16
Problématiques en cours	17
Benchmark Concurrentiel	17
1)Architecture du projet (front-end)	18
2)Conventions de nommage	18
3)Lien Figma ↔ Code	18
4)Stack technique (2024)	18
5)Branches GitHub	19
6. Outils à maîtriser	19
7. Documentation et mots-clés	19
8. Objectifs immédiats (fin 2024 – début 2025)	19
Branches Git	20
État du code	20
Outils obligatoires	20
Traductions	20
Base Documentation / À savoir	20
VI Etat des lieux de Socializus 2024	21
1)Contexte du projet	21
2)Les environnements	21
Trois environnements distincts sont en place :	21
3)Organisation des repositories	21
4)Points importants à retenir	21
Repositories GitHub	22
3 environnements à connaître	22
Nommage des dossiers	22
Figma	22
Règle de travail	22
VII S'installer & accéder aux outils	22
Figma	23
GitHub	23
Comptes dev	23
IX Le protocole d'arrivée	23
Routine de début de journée	23
Nommage des branches	23
Utilité des branches quotidiennes	24
Live Share (VS Code)	24
Historique (tableur)	24
Conflits Git	24
XI Journée type de travail	25
Contexte et objectif	25

1. Organisation des réunions quotidiennes (Daily / Débrief)	25
2. Accès aux dépôts GitHub	25
3. Différences entre les versions du code	25
Code 2023 Code 2024	25
4. Conventions de code	26
5. Processus de commits GitHub	26
6. Première mission du stagiaire	26
Mots-clés de la séance :Croissant · Actualisation	26
XX - Coder avec l'IA	28
Préparation du plan de travail	28
Protocole de connection	28
XII - Expression du besoin (EDB), IA et restructuration	26

I - LA JOURNÉE D'INTÉGRATION

Titre de la vidéo : Journée d'intégration -Etape 0- 1juin 2026 :

<https://www.youtube.com/watch?v=907dNbjtOcQ>

Titre de la vidéo : Journée d'intégration 1,2,3 - 1 juin 2026 :

<https://www.loom.com/share/856e703d62aa4e3688a0184fe2417cb5>

Titre de la vidéo : Journée d'intégration 4,5,6 - 16 juin 2026 :

<https://www.loom.com/share/82f0a7fb33024dc9ac521fdf96e7fe0a>

Titre de la vidéo : Journée d'intégration (part 7,8,9) - 22 juin 2026 :

<https://www.loom.com/share/e9bf9539cd6e4267b2e2c3b98220d014>

1. Principe des réunions asynchrones

L'organisation repose sur un système de **réunions asynchrones** (vidéos enregistrées) plutôt que des réunions en temps réel. Cela permet :

- D'éviter les répétitions pour le maître de stage.
- À chaque stagiaire de visionner les vidéos à son rythme (vitesse $\times 1,2$ ou $\times 1,5$ possible).
- De travailler à distance sans contrainte horaire.

Règle fondamentale : Visionner les vidéos en étant concentré et reposé.

2. Outils de communication

a - WhatsApp

- Outil principal de messagerie instantanée professionnelle.
- À utiliser sur **ordinateur** (**OBLIGATOIRE** web.whatsapp.com).
- Permet d'indiquer les **mots-clés** à chaque vidéo visionnée *provisoirement avant le conducteur*
- Permet de **tamponner les horaires** de début et fin de journée.
- Permet de **poser des questions par écrit** de façon concise.

b - Téléphone

- Réservé aux **urgences uniquement** (équivalent de frapper à la porte du bureau).
- À utiliser si WhatsApp reste sans réponse après plusieurs tentatives.

c - Google Meet

- Salle de réunion virtuelle permanente : les stagiaires y sont connectés pendant toute la journée de travail (**télétravail**, pas distanciel).
- Pas de mute pendant les réunions en cours.

- Toujours partagé **tout l'écran** (pas seulement un onglet, ni une fenêtre) lors du partage d'écran.
- Se présenter par son **prénom** avant de prendre la parole.

3. Protocoles de communication et de travail quotidien en télé présentiel

Situation	Action
Début de journée	Envoyer sur WhatsApp : "Bonjour, je suis dans Google Meet, je commence ma journée"
Problème technique ou pause	Signaler votre absence sur whatsapp
Pause déjeuner	Écrire "pause déjeuner" dans whatsapp
Avant la Question	La formuler par écrit sur whatsapp (oblige à être concis et structuré)
Prendre la parole	Se présenter par prénom + sujet en 3-4 mots dans Google Meet si le responsable est présent
Pas de réponse	Envoyer un message WhatsApp
Toujours pas de réponse	Appeler par téléphone (urgences principalement)
Fin de journée	Envoyer sur WhatsApp le total d'heures de vidéo regardées dans la journée

a - Règles importantes

- Ne jamais rester en mute pendant une réunion
- Toujours partagé tout l'écran (pas seulement un onglet) lors des partages d'écran
- Ne pas cliquer sur des liens dans un onglet actif — toujours ouvrir un nouvel onglet vierge pour ne pas quitter la réunion
- Travailler en mode moitié écran vidéo / moitié écran notes
- Ne jamais personnaliser les couleurs ou l'interface de l'utilisateur Chrome pro

4. Environnement de travail — Chrome et compte professionnel

Utilisation d'un utilisateur Chrome “partagé”

- Utiliser un **utilisateur dédié** “...@gmail.com” dans Chrome (distinct du compte personnel).
- il permet d'avoir accès au conducteur mais pas à la documentation

- Se connecter avec l'adresse email pro fournie par le maître de stage ou exceptionnellement la créer .
- Activer la **synchronisation** lors de la configuration.
- Répondre **oui à toutes les questions** posées par Google (compte partagé , pas personnel).
- Ne **jamais personnaliser** l'interface (couleurs, thèmes, extensions non autorisées) et installer les extensions demandés dans le formulaire comme le thème foxtro 1080p "renard"
- Ne jamais se connecter avec son téléphone sur ce utilisateur

Utilisation d'un utilisateur Chrome "pro"

- **utilisateur dédié** dans Chrome (distinct du compte personnel et du compte partagé).
- Se connecter avec l'adresse email pro fournie @nomdedomaine.fr ou nomdedomaine.org par le responsable.
- Activer la **synchronisation** lors de la configuration.
- Répondre **oui à toutes les questions** posées par Google (compte pro , pas personnel).
- Ne **jamais personnaliser** l'interface (couleurs, thèmes, extensions non autorisées).
- Conserver la couleur d'environnement attribuée (ex. jaune pour le pôle RH, bleu/vert pour le dev).
- Ne jamais se connecter avec son téléphone sur ce utilisateur

Règles importantes

- **Ne jamais se connecter avec un téléphone** pour les comptes pro et partagé.
- **Ne jamais mélanger** comptes personnels et professionnels et partagé.
- Préférer le **Google Connect** (connexion via Google) plutôt que des mots de passe manuels sur les sites annexes.
- Sauvegarder les mots de passe **dans l'utilisateur Chrome pro uniquement**.

Adresse email de récupération

- Utilisée à la place du code de confirmation envoyé sur téléphone.
- Format : xxxx@gmail.com (fournie par le responsable).

5. Gestion des mots de passe

- Créer un **dossier "MDP"** dans les favoris Chrome pour stocker le mot de passe
- Le mot de passe inclut la **date de création** pour en assurer le suivi et son nom prénom.
- Le modifier à la prise de poste et communiquer le nouveau mot de passe au maître de stage.

- Ne stocker **aucun mot de passe personnel** dans l'espace professionnel.

6. Prises de notes — Google Drive et Google Docs

- **Jamais de Word** : utiliser exclusivement **Google Docs**.
- Créer un dossier dans Google Drive intitulé *"Partage avec le maître de stage"*.
- Partager ce dossier avec les adresses : association@... et jpi@socialisus.org.
- Tout document créé dans ce dossier sera automatiquement partagé.
- Prendre des notes en **mode moitié-écran** : une moitié pour la vidéo, l'autre pour les notes.

7. Le conducteur d'intégration

Le **conducteur** est un tableau de suivi (format tableur) qui liste toutes les vidéos d'intégration. Pour chaque vidéo, le stagiaire doit renseigner :

- La **durée** de la vidéo.
- Le ou les **mots-clés** identifiés (ou une croix rouge si aucun mot-clé).
- Une **évaluation de pertinence** (croix rouge si la vidéo est jugée inutile ou redondante).
- La somme des durées en fin de journée via une **formule Excel** (`=SOMME(C2:CX)/60`).

Le conducteur doit être **synchronisé avec la documentation** : toute vidéo présente dans l'un doit l'être dans l'autre.

8. La documentation

- La documentation est le **support de référence** : elle accompagne et complète les vidéos.
- Elle est organisée par sommaire : commencer par lire le sommaire avant de visionner les vidéos.
- Les vidéos et la documentation doivent être **en parfaite synchronie** (même contenu, même ordre).
- Le stagiaire contribue à **signaler les décalages** entre conducteur et documentation.

9. Outils techniques complémentaires

Google Remote Desktop

- Extension Chrome permettant de **prendre le contrôle de l'écran** à distance pour guider.

- Télécharger depuis le Chrome Web Store : rechercher "*Google Remote Desktop extension*".
- Le code de partage est **provisoire** : il expire après chaque session.
- Épingler l'extension dans Chrome pour y accéder rapidement.

Loom

- Application d'enregistrement vidéo installée dans l'environnement Chrome professionnel.
- Peut être utilisée pour des **réponses vidéo asynchrones**.
- Se connecter avec le compte professionnel uniquement (pas de création de compte personnel).

Winscribe / VPN (Hola)

- Permet de masquer l'adresse IP personnelle lors de manipulations sensibles.
- Installé automatiquement dans l'environnement Chrome professionnel.

10. Posture et engagement attendus

- Le stage ne donne pas lieu à une rémunération : l'objectif pour le stagiaire est d'obtenir une **bonne évaluation** et une **recommandation solide**.
- L'implication est le facteur déterminant : un stagiaire peu impliqué apporte moins de valeur qu'un outil d'IA gratuit.
- Le maître de stage est disponible mais **ne peut pas répondre immédiatement** à tout moment : respecter les protocoles de communication.
- Ne pas modifier, personnaliser ou altérer l'environnement de travail partagé sans autorisation.
- Signaler les fins de journée avec le temps total de vidéos visionnées.

II - DOCUMENTATION, ORGANISATION ET IA 2026

Pour remettre en état la documentation il faut utiliser et rapatrier progressivement dans le conducteur et dans la documentation ici les vidéos qui sont listés [dans cette documentation](#)

Amélioration doc et conducteur - 2026 Compilation

<https://www.loom.com/share/d61375b9cb4444208abbaf93188ec36e>

Titre de la vidéo: la doc et le conducteur - 13 May 2026

<https://www.loom.com/share/756d6261ccc9422ba>

Titre de la vidéo:Organisation de la doc restructurée avec IA - 2026 Part 1c

<https://www.loom.com/share/7781a4c0ffeb4120943aee5205ac568a>

Titre de la vidéo:Organisation de la doc restructurée avec l' IA part 2,3,4,5,6 - 8 May 2026

<https://www.loom.com/share/c2a8fb1b3d4949428ce3ce837fa51a1b>

1. Objectif de la documentation

- Créer une documentation centralisée et exploitable pour les développeurs.
- Réduire la dépendance aux échanges directs.
- Permettre une intégration plus autonome des nouveaux développeurs.
- Identifier les éléments obsolètes et les faire évoluer.

2. Organisation générale du projet

- Le projet repose sur un tableur principal servant de centre de pilotage.
- Le tableur contient : historique, front, back, tests, bugs, assets et organisation des fichiers.
- Les vidéos servent de support complémentaire mais ne remplacent pas la documentation écrite.
- La doc et le conducteur doivent être en permanence synchronisés : toute vidéo présente dans l'un doit l'être dans l'autre, avec le bon lien et le bon numéro de page.
- Quand une vidéo est nettoyée ou raccourcie, la doc et le conducteur doivent être mis à jour en conséquence.
- Les éléments incorrects, liens cassés ou titres mal nommés doivent être corrigés de manière proactive, sans attendre.
- Chaque thématique (ex : documentation, architecture du code) doit former un bloc regroupé dans le conducteur, comme un futur chapitre de cours.
- Le numéro de page de la doc doit toujours être renseigné dans la colonne correspondante du conducteur.

3. Utilisation de l'IA (Claude)

- Claude est utilisé en mode projet (5 projets maximum disponibles par compte).
- Chaque projet correspond à une grande thématique (ex : Journée d'intégration, Architecture du code, Documentation...).
- À l'intérieur d'un projet, on peut ouvrir plusieurs discussions et compiler plusieurs vidéos.
- Le projet doit être configuré avec un brief initial (prompt système) précisant le contexte, le rôle de l'étudiant, et les règles de mise en forme souhaitées (tableaux propres, pas de lignes de séparation, format copiable dans Google Docs).
- L'IA produit un résumé textuel structuré, qu'on améliore en lui envoyant les vidéos les unes après les autres.
- On utilise le compte professionnel Chrome pour accéder à Claude, jamais le compte personnel.

III Architecture du code 2026

Architecture -Extra 1 - 11 mai 2026

<https://www.loom.com/share/93bf7bff314241a99535080b9c1a193e>

6) Historique et suivi

- L'historique doit être rempli ligne par ligne.
- Les développeurs juniors peuvent documenter les modifications réalisées.
- Les développeurs expérimentés valident ou corrigent les explications.
- Les éléments incorrects peuvent être signalés en orange.

7) Front-end

- Le front est organisé par pages, composants et styles.
- Les fichiers relatifs à un écran doivent rester regroupés.
- Les composants partagés doivent être clairement identifiés.

8) Back-end

- Le backend reste encore partiellement documenté.
- Les routes API doivent être progressivement ajoutées.
- Postman est utilisé pour tester et documenter les routes.

9) Tests

- Les tests doivent être réalisés sur plusieurs environnements.
- Tests localhost.
- Tests Android.
- Tests iOS.

- Tests serveur de test.
- Tests 4G et comportements réels utilisateurs.

12)Assets et ressources

Les assets doivent être normalisées.

- Préférence pour des exports 512x512.
- Attention aux exports Figma et Flaticon.

13)Intégration développeur

- L'intégration demande plusieurs jours de prise en main.
- Les vidéos peuvent être accélérées pour gagner du temps.
- Les nouveaux développeurs doivent signaler les contenus inutiles ou obsolètes.
- L'objectif est d'améliorer progressivement la qualité de la documentation.

10)Gestion des bugs

- Les bugs doivent être décrits clairement.
- Chaque bug doit contenir : auteur du signalement, état, correctif et environnement.
- Le suivi des bugs est lié aux campagnes de tests.

3)Évolutions par rapport à 2024

- Passage progressif vers une architecture plus propre et modulaire.
- Standardisation des noms de fichiers en PascalCase.
- Externalisation des styles CSS.
- Découpage des écrans en composants réutilisables.
- Suppression progressive des anciennes conventions obsolètes.

4)Convention de nommage

- Utilisation obligatoire du PascalCase.
- Suppression des anciens formats comme style.web.js.
- Exemple attendu : StyleWeb.js / StyleMobile.js.

5)Structure des écrans

- Chaque screen doit rester lisible et court.
- Les gros fichiers de plusieurs centaines de lignes doivent être évités.
- Les composants doivent être séparés.
- Les styles doivent être externalisés.

11)Authentification et sécurité

- La confirmation email devient obligatoire.
- Les anciens comportements de 2023 ne sont plus valables.
- Les rôles utilisateurs influencent les droits disponibles.

14)Résumé des changements majeurs

Sujet	Ancienne logique	Nouvelle logique
Architecture	Web/Mobile très séparés	Architecture plus modulaire
Styles	style.web.js	PascalCase + externalisation
Module	un dossier pour clarifier un ensemble important composé de section et de screen	
Section	un dossier pour clarifier un ensemble important composé de screen et screens	
Screens	un dossier pour clarifier un ensemble important composé de screen	Composants séparés
Screen	un dossier pour clarifier une page décomposé en style.ts et Code.tsx	
Email	Confirmation parfois facultative	Confirmation obligatoire
Documentation	Très brute	Tri et clarification
Tests	Peu structurés	Tests segmentés

15) Restructuration de l'architecture du code en mode responsive.

RÉSUMÉ :

Quelques réformes ont été mises en place dans l'architecture du projet afin d'améliorer l'organisation du code et la gestion du responsive.

Une séparation stricte des styles est désormais demandée avec une structure claire :

- `style-mobile`
- `style-tablette`
- `style-desktop`

Chaque module devra également contenir un [README.md](#) afin d'expliquer le fonctionnement des screens, la logique métier et les éléments importants du code. L'objectif est de rendre le projet plus compréhensible et plus simple à maintenir pour tous les développeurs.

L'organisation des fichiers sera désormais faite par modules, puis découpée en screens et composants réutilisables lorsque nécessaire. Une attention

particulière est portée au nommage des fichiers et des dossiers afin d'éviter les ambiguïtés entre les versions mobile, tablette et desktop.

Le responsive introduit également une nouvelle logique d'organisation pour les pages desktop, certaines étant composées de plusieurs frames ou sections regroupées dans une même vue.

Cette restructuration vise à standardiser toute l'architecture du projet avec une structure identique sur l'ensemble des modules, un code plus lisible, une meilleure séparation des responsabilités et une maintenance plus simple sur le long terme.

LA VIDEO : <https://www.loom.com/share/4e6ffe6667df46d8a8c7d7fa0a20779e>

Arborescence type

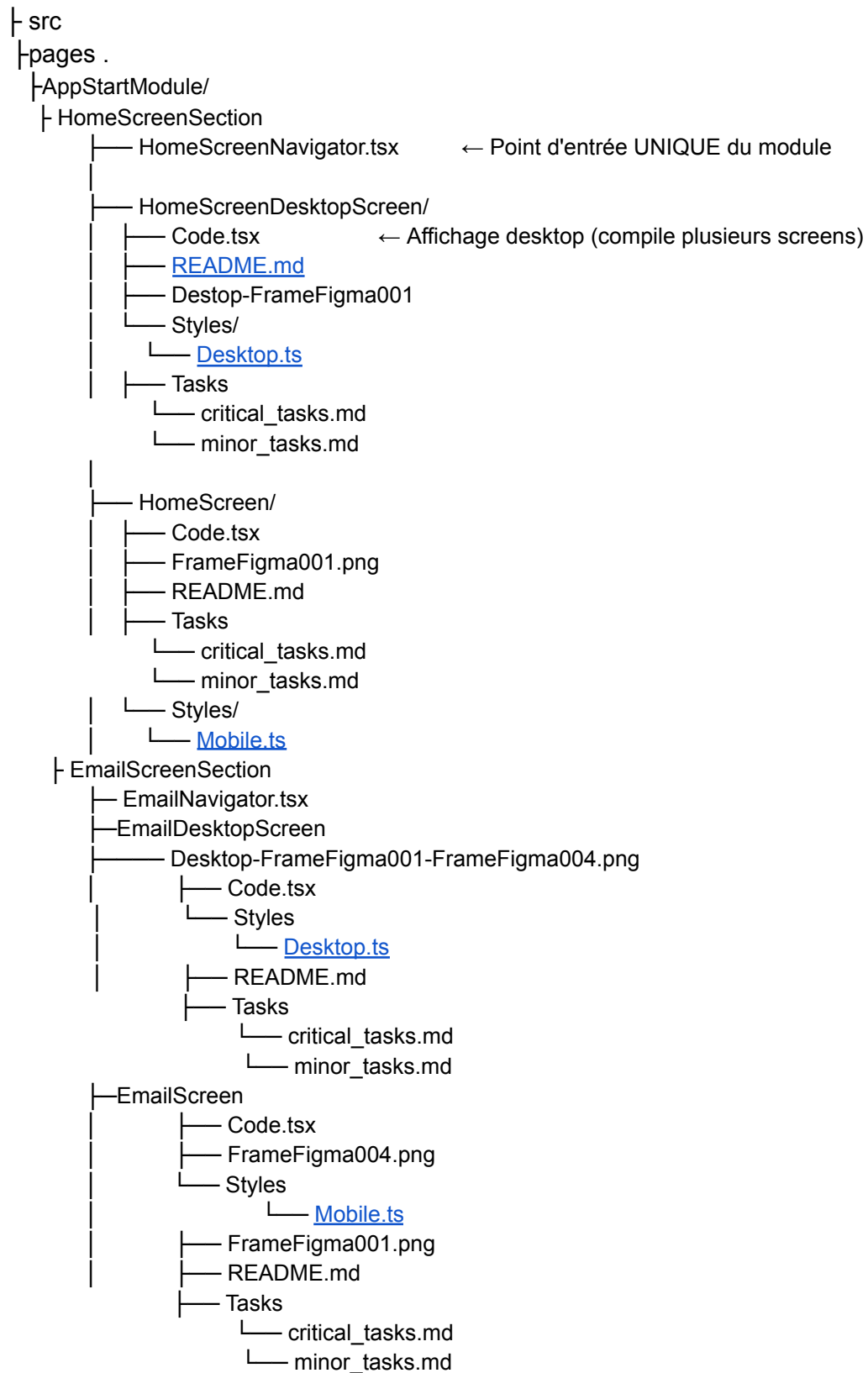
frontendweb

- src
- pages

....

CalendarModule/

- CalendarNavigator.tsx ← Point d'entrée UNIQUE du module
- CalendarDesktopScreen/
 - Code.tsx ← Affichage desktop (compile plusieurs screens)
 - [README.md](#)
 - Destop-FrameFigma020-frameFigma021.png
 - Styles/
 - Desktop.ts
- ActivitesListScreen/
 - Code.tsx
 - FrameFigma020.png
 - README.md
 - Tasks
 - critical_tasks.md
 - minor_tasks.md
 - Styles/
 - Mobile.ts
- MyActivitiesScreen
 - MyActivitiesLikedScreen
 - FrameFigma021c.png
 - Code.tsx
 - Styles
 - Mobile.ts
 - MyActivitiesNavigator.tsx
 - MyActivitiesOrganizedScreen
 - FrameFigma021c.png
 - Code.tsx
 - Styles
 - Mobile.ts



V Compilation 2026

Welcome dev - Compilation 2026

<https://www.loom.com/share/cb7867e8531647fda6ff6f23778c6875>

Contexte & Objectif

Ce projet s'inscrit dans le cadre de l'association **ONDRH**, qui développe l'application mobile et web **Socialisus**. L'objectif est d'intégrer de nouveaux développeurs (souvent en TDM ou bénévoles) dans un environnement de travail structuré, documenté par vidéos et conducteurs écrits, pour pallier le fort turnover et réduire le temps d'explication.

Environnement de travail & Outils

- **Conducteur** : document de 120–130 pages servant de guide pas-à-pas, accompagné de vidéos explicatives. En cours de nettoyage et synthèse.
- **Tableur** : micro-application de suivi des tests, bugs, rôles, historiques et expressions de besoin.
- **GitHub Desktop** : outil de versioning privilégié (interface graphique préférée aux lignes de commande).
- **Figma** : utilisé pour illustrer l'expression du besoin et les maquettes.
- **DB Compass** : outil de gestion de base de données (accès restreint selon liste blanche).
- **Compte développeur dédié** (Dev2, Dev3...) : accès Google + Figma + documents partagés + favoris préconfigurés.

Architecture du Code

- **Frontend / Backend** séparés en dossiers distincts dans le repository.
- Chaque dossier contient : le **code principal**, le **style web** et le **style mobile**.
- Migration en cours vers **TypeScript** et une architecture plus propre (2024).
- Système de **mots-clés** à envoyer via WhatsApp pour valider la lecture des vidéos.

Stratégie Mobile & WebView

L'application cible **trois OS** : iOS, Android et **HarmonyOS (Huawei)**. Pour couvrir Huawei sans recoder l'application entièrement, la stratégie retenue est la **WebView** :

un navigateur open source qui affiche l'application web en simulant une app native (sans barre de navigation visible).

- Avantage : code open source facilement trouvable, compatible tous stores (Play Store, App Store, AppGallery).
- Situation actuelle : **deux versions Android** coexistent — l'une compatible Android 8–12, l'autre Android 13–14.
- L'application a été refusée plusieurs fois par Google, notamment pour des problèmes d'UX (bouton "Start" affiché deux fois selon la langue).

Problématiques en cours

Sujet	État
Migration serveur .eu → socialisus.app / .us	Bloquée (affichage IP non fonctionnel)
WebView Huawei (AppGallery)	En recherche d'open source
Restriction d'âge (+18) sur Android	À revoir dans les réponses store
Tests unitaires	Non encore implémentés
Benchmark Fremek / Knock	En cours (captures d'écran)
Chat natif iOS/Android	En développement

Benchmark Concurrentiel

Un benchmark détaillé de **Frlmake** a été réalisé (captures d'écran du parcours d'inscription). Points à intégrer dans Socializus : demande de date de naissance dès l'inscription, question parrain reformulée ("j'ai un parrain oui/non"), validation du format du prénom/nom.

RÉSIDU À ÉLIMINER

La doc principale est synchronisée avec le de **conducteur** : une liste ordonnée de vidéos et pages à consulter.. Chaque vidéo contient un **mot-clé** à indiquer sur le conducteur pour confirmer que la vidéo a été vue.

Un **tableur** centralise : rôles, expression du besoin, bugs, tests et historiques. Pas de tests unitaires en place pour l'instant.

VI PRÉSENTATION DU PROJET SOCIALIZUS

PRÉSENTATION DU Projet SOCIALIZUS - 16 September 2024

<https://www.loom.com/share/3cfdcc4832654ee2b2bcc051eef374a6>

1) Architecture du projet (front-end)

Le dossier **src** contient tous les éléments classés en minuscules : **assets**, **components**, **pages**, **redux**, **constants**, **contexts**, **data**, **jsons**. À partir des composants, tout passe en **PascalCase**. Les composants sont classés par localisation (sections, screens). Chaque screen contient le code sans style, puis le style web et le style mobile séparément.

2) Conventions de nommage

- Dossiers racine en **minuscules**, Page et composants en **PascalCase**
- Chaque constante ou composant doit porter un nom **clair et unique**, compréhensible hors contexte
- Les éléments vectoriels sont dans un dossier **Vectors** (fichiers **.svg**), les images dans un dossier **Images**
- Les frames Figma sont nommées **fremefigma050a** (tout attaché) pour permettre des recherches précises dans le code

3) Lien Figma ↔ Code

Chaque page de code doit référencer la frame Figma correspondante (ex. **fremefigma050a**). Cela permet de retrouver facilement tous les composants liés à une frame via une simple recherche dans le projet.

4) Stack technique (2024)

- Migration de **JavaScript** vers **TypeScript** (en cours, des **any** restent à corriger)
- Remplacement du **Context** par **Redux** pour la gestion du state
- Application disponible en **mobile** (iOS/Android) et **web** (responsive en cours)
- Traductions gérées via des fichiers **JSON** (noms de variables non traduits volontairement pour éviter les conflits avec Google Translate)

5) Branches GitHub

Branche	Description
main	Code stable (~4 mois), synchronisé avec socialisus.us (site de test)
dev	Code le plus récent, en TypeScript, en cours de finalisation

Le site de **production** est **socialisus.app**.

6. Outils à maîtriser

- **GitHub Desktop** — gestion des pushes (format date : **AAAA-MM-JJ HH:MM**), résolution de conflits
- **Figma** — lecture des frames, récupération des assets
- **MongoDB Compass** — connexion via string, nettoyage BDD, gestion de la whitelist IP
- **Environnement Chrome** (Chromebook-like) — outil de travail principal

7. Documentation et mots-clés

La doc est l'élément central (pas le conducteur vidéo). Des **mots-clés cachés** y sont intégrés pour vérifier que les stagiaires l'ont bien lue. Mots-clés de cette vidéo :

- **chapitrage architecture**
- **travailler pour vous**
- **environnement chrome**
- **global**
- **communication**

8. Objectifs immédiats (fin 2024 – début 2025)

- Corriger les **any** TypeScript restants
- Nettoyer les styles (supprimer les lignes redondantes)
- Finaliser le responsive web
- Ajouter des commentaires dans le code là où c'est nécessaire
- Mettre en place des tests automatisés
- Fusionner vers **dev**
- Améliorer la note de l'application sur le store Android

Branches Git

- `main` = code synchronisé avec socialisus.us ou socializus.eu (site de test, JavaScript)
-

État du code

- TypeScript en cours, nombreux `any` restants à corriger
- Styles à nettoyer : supprimer les lignes CSS inutiles
- Pas de tests automatisés en place pour l'instant
- App web et mobile synchrones

Outils obligatoires

- GitHub Desktop (commits avec format date : `AAAA-MM-JJ HH:MM`)
- Figma (Google connect avec compte Dev fourni)
- MongoDB Compass (connexion via string + IP à ajouter en whitelist)
- Environnement Chrome (session dédiée)

Traductions

- Fichiers `json` par langue dans le dossier `jsons`

Variables non traduites par Google Translate

Base Documentation / À savoir

VI Etat des lieux de Socializus 2024

État des lieux Socializus 2024 part2 — 22 avril 2024

<https://www.loom.com/share/bc6459f6969047a080e8dd3735fa36bc>

État des lieux de 2024 avec simplifications 📄

<https://www.loom.com/share/85578550085949518d0f77de571b5af4>

1)Contexte du projet

Le projet dispose de plusieurs repositories GitHub, répartis entre le front-end et le back-end, et organisés au sein d'une structure en cours de clarification.

2) Les environnements

Trois environnements distincts sont en place :

- **Serveur de test** — pour les premiers essais et bidouillages
- **Serveur de "gros test"** (anciennement nommé *Socialise.us*, désormais renommé *Erasmus Evans*) — pour des tests plus poussés
- **Serveur de production** — environnement final, avec ses propres noms de domaine

3) Organisation des repositories

Les repositories sont regroupés dans une organisation GitHub appelée **App Socialisme Centre**. On y trouve notamment deux repositories principaux : le serveur test 2023 et le serveur gros test 2020. Le repository 2023 sera éventuellement archivé ou supprimé.

4) Points importants à retenir

- Le code **2023** est plus simple visuellement, tandis que le code **2024** est plus complexe. Il est utile de comparer les deux pour comprendre l'évolution de l'application.
- Le nom des branches contient un tiret (et non un point) pour éviter qu'elles s'affichent comme des applications sur Mac.
- **Règle de travail** : toujours commencer par travailler sur l'environnement de test avant de passer aux autres.

RÉSIDU À ÉLIMINER

Repositories GitHubbloom

- Plusieurs repositories existent depuis le début du projet
- Organisation principale : **App Socialisus Centre** → contient 2 repos actifs : serveur test 2023 et serveur gros test 2024
- Le code 2023 sera basculé vers **Erasmus Evans** (environnement de bidouillage/preuves)
- À terme : supprimer ou archiver le repo 2023 devenu inutile

3 environnements à connaître

- Serveur de **test** → pour les expérimentations
- Serveur **gros test** → tests plus poussés
- Serveur de **prod** → production réelle

Nommage des dossiers

- Mettre un tiret dans le nom du repo pour éviter qu'il s'affiche comme une app sur Mac (ex: `socialisus-app` et non `socialisus.app`)

Figma

- Le code 2024 est visible sur Figma → visuellement plus complexe que 2023
- Utile pour comparer les deux versions et comprendre l'évolution

Règle de travail

- Toujours tester d'abord sur le serveur de test avant de pousser en prod

VII S'installer & accéder aux outils

Les mots de passe (part2) — 29 juin 2023

<https://www.loom.com/share/cd07c048228a42378ed16be59f993a9a>

Figma

- Connexion via **Google Connect** avec le compte dev fourni
- Accès en mode **lecture seule** par défaut (pas édition)
- Si accès manquant → demander au responsable d'ajouter `dev11@socializus.org` via "Partager"

GitHub

- Pas de Google Connect disponible → connexion manuelle obligatoire
- Changer son mot de passe dès l'arrivée (mot de passe envoyé via WhatsApp)
- Ne pas autoriser la mémorisation du mot de passe GitHub dans le navigateur
- Identifiants : email `dev11@...` + mot de passe fourni

Comptes dev

- Comptes nommés par numéro (`dev1`, `dev2`... `dev11`) car présence éphémère (stage)
- Permet de réutiliser les comptes sans changer les accès à chaque rotation

IX Le protocole d'arrivée

Protocole d'arrivée — 21 avril 2023

<https://www.loom.com/share/895ab4c494194486b68c99549c127219>

Routine de début de journée

- Lancer **GitHub Desktop** en premier, tout le reste fermé (VS Code inclus)
- Créer une nouvelle branche avec le format : `AAAA-MM-JJ-HH-MM-dev1`
- Publier la branche en fin de journée (pas obligatoire le matin)

Nommage des branches

- Format date inversée : année → mois → jour → heure → initiales/dev
- Exemple : `2023-04-21-10-05-dev4`
- Permet un classement chronologique automatique

Utilité des branches quotidiennes

- Permet de revenir en arrière sur le code d'un jour précédent
- Fonctionne comme une mémoire tampon sans écraser le `main`
- Si non publiée : accessible via le **stash** (pile en attente)
- Discard changes si les modifications ne sont pas pertinentes

Live Share (VS Code)

- Partager le code via Live Share → envoyer le lien dans la discussion
- Accès en écriture pour tous par défaut
- Permet de suivre visuellement où est chaque développeur dans le code
- Le simulateur ne tourne que sur la machine hôte (pas sur les machines des autres devs)

Historique (tableur)

- Remplir en début de journée : date, prénom, ce qu'on prévoit de faire
- Permet d'éviter les **conflits** entre devs qui travailleraient sur les mêmes fichiers
- Règle : une personne = une page, si deux personnes sur la même page → communication obligatoire via Google Meet

Conflits Git

- Ne jamais fusionner une branche sans validation
- Back-end et front-end = dossiers séparés → jamais de chevauchement

XI Journée type de travail

1ère journée et challenge — 5 mars 2024

<https://www.loom.com/share/b0c5196d38ad42f688e2a49c282f3545>

Contexte et objectif

L'objectif est d'introduire les outils, processus de travail et conventions de code utilisés dans le projet.

1. Organisation des réunions quotidiennes (Daily / Débrief)

- Chaque matin, une réunion courte de débrief (scrum / brainstorming) est organisée.
- Le stagiaire doit **envoyer chaque matin un lien** vers sa copie du document du jour, déposée dans le dossier partagé prévu à cet effet.
- Le document quotidien contient trois sections : "**Qu'est-ce que j'ai fait hier ?**", "**Qu'est-ce que je fais aujourd'hui ?**", et une checklist des missions.
- Le partage d'écran sur Google Meet fonctionne en mode "le dernier qui partage prend la main" — inutile de demander à l'autre de couper son partage.

2. Accès aux dépôts GitHub

- L'organisation GitHub s'appelle **AbsoSocialiseuse**.
- Il existe plusieurs dépôts selon les années : **2021, 2023, 2024**.
- Le stagiaire commence en **lecture seule sur le code 2021**, puis **lecture/écriture sur le code 2023** pour s'entraîner.
- L'accès au code 2024 se débloque après validation des compétences de base.

3. Différences entre les versions du code

	Code 2023	Code 2024
Framework	React Native avec Expo	React Native sans Expo
Déploiement serveur	Scripts personnalisés	Docker
Statut	En lecture/écriture pour tests	Accès progressif

4. Conventions de code

- Nommage en **PascalCase** obligatoire.
- Les dossiers de pages s'appellent **screens** (ex. **CreateProfileStep4Screen**).
- Les éléments parents qui gèrent la navigation entre étapes s'appellent **Navigator** (et non Screen).
- Les étapes sont préfixées par leur numéro (**1_**, **2_**...) pour un classement chronologique correct.
- Jamais de texte affiché en dur dans le code : tout passe par des **variables/objets JSON** pour gérer la **multilingue** (français, anglais, russe...).

5. Processus de commits GitHub

- Format du commit : **AAAA-MM-JJ - HH:MM - [initiales du développeur]**
- Un **historique structuré par chemin de fichier** est tenu dans un tableur dédié (un tableur 2023, un tableur 2024).
- Pour chaque commit, trois rôles sont identifiés : **celui qui pousse**, **celui qui explique** (note l'historique), **celui qui vérifie**.
- Les explications de commits doivent être rédigées en propre — **pas de copier-coller de ChatGPT** comme description.

6. Première mission du stagiaire

1. Lancer le **front-end web** du code 2023.
2. Lancer le **front-end mobile** du code 2023 (simulateur).
3. Reproduire les deux sur le **code 2024**.
4. Compléter la documentation si des éléments manquants sont identifiés.

Mots-clés de la séance :Croissant · Actualisation

Scrum Daily Meeting — 23 novembre 2023

<https://www.loom.com/share/3dedae176bdc43b088524844ab932603?sid=c5753390-004a-4ad8-af56-63339c0bb1db>

Les références du code - 2026 Compilation

<https://www.loom.com/share/c06ddaa5adc6475bbacb94be248d0210>

XX - Coder avec l'IA

Préparation du plan de travail

<https://www.loom.com/share/715b0655141746cb9f324c6c339d9dba>

Protocole de connection

<https://www.loom.com/share/c7bada2d111041b8b6c79485d4163e96>

<https://www.loom.com/share/04a79ad03b1647cc89a2fb00b0c07305>

<https://www.loom.com/share/fbc37c29644942a18eca8782b02afc0d>

XII - Expression du besoin (EDB), IA et restructuration

<https://www.loom.com/share/d950a30db4ab4cc1a510a98a71925fb4>

1. Objectif de l'expression du besoin

L'expression du besoin (EDB) sert à :

- * décrire les fonctionnalités du projet,
- * organiser les modules,
- * clarifier les comportements utilisateurs,
- * aider les développeurs,
- * suivre l'avancement du projet.

Deux versions principales sont utilisées :

- * V1 : ancienne version brouillon,
- * V5 : version restructurée avec l'aide de l'IA.

2. Utilisation de l'IA

L'IA est utilisée pour :

- * restructurer les informations,
- * créer des chapitrages,
- * comparer les versions,
- * détecter les oublis,
- * améliorer la documentation.

L'IA doit toujours être vérifiée et corrigée par les stagiaires.

3. Comparaison V1 ↔ V5

Le travail principal consiste à :

- * comparer les deux versions,
- * vérifier les lignes transférées,
- * détecter les oublis,
- * supprimer les doublons,
- * améliorer la cohérence.

Le travail manuel doit être fait avant de demander une vérification à l'IA.

4. Analyse Figma

Les maquettes Figma servent de référence visuelle du projet.

Le stagiaire doit :

- * analyser les écrans,
- * identifier les fonctionnalités visibles,
- * vérifier si elles existent dans l'EDB,
- * ajouter les éléments manquants.

5. Notifications

Le projet contient plusieurs catégories de notifications.

a - App Notifications

Notifications automatiques de l'application :

- * activité presque complète,
- * rappels,
- * nouveautés,
- * informations système.

b - Members Notifications

Interactions entre utilisateurs :

- * like de profil,
- * visite de profil,
- * anniversaires,
- * commentaires,
- * interactions sociales.

c - Activities Notifications

Notifications liées aux activités :

- * nouvelles activités,
- * activités proches,
- * organisateurs rencontrés,
- * propositions d'activités.

6. Système de statuts

Des icônes permettent de suivre l'état des fonctionnalités.

-  À faire
-  Bug
-  En cours
-  À vérifier
-  Validé
-  Régression
-  Essentiel
-  Fonctionnel

7. Méthode de travail

Étapes principales :

- * comparer V1 et V5,
- * analyser Figma,
- * ajouter les éléments oubliés,
- * corriger les incohérences,
- * utiliser l'IA pour vérifier,
- * restructurer la documentation.

Mots clé : Mission amelioration EDB5; Analyse Figma